go语言基础

```go
package main // 每个文件必须以 package 开头，main 表示可执行程序入口

import "fmt" // 导入标准库中的 fmt 包，用于格式化输入输出

func main() { // main 函数是程序入口
    fmt.Println("Hello, World!")
}
```

定义变量

```go
var a int = 10       // 显式声明变量
var b = 20           // 类型推导
c := 30              // 简短声明（仅限函数内部使用）
var a, b int


var x int = 10
var y float64 = float64(x) // 必须显式转换
```

定义常量

```go
const PI float64 = 3.1415926
```

结构体

```go
type Person struct {
    Name string
    Age  int
}

p1 := Person{"Alice", 25}                // 按顺序初始化
p2 := Person{Name: "Bob", Age: 30}       // 指定字段名初始化
p3 := Person{}                            // 使用零值初始化：Name="", Age=0
```

面向对象

```
func (p Person) SayHello() {
    fmt.Printf("你好，%s! \n", p.Name)
}

// 调用方法
p1.SayHello()
```

for循环案例

```
    limit := min(3, len(votes))
    for i := 0; i < limit; i++ {
        fmt.Printf("%s:%d\n", votes[i].ID, votes[i].Count)
    }


// min returns the minimum of two integers
func min(a, b int) int {
    if a < b {
        return a
    }
    return b
}
```

go语言：切片和字典

slice（切片）

初始化切片

```
s := []int{1, 2, 3}            // 直接初始化
s := arr[:]                    // 从数组创建切片
s := make([]int, 0, 5)         // 创建空切片，容量为5
```

动态追加元素

```
s = append(s, 4) // 自动扩容
```

截取切片

```
sub := s[1:3] // 左闭右开区间，索引1~2元素
```

零值和 nil 判断

```
var s []int // nil 切片，注意nil切片和空切片的区别 s:=[]int{} s!=nil
if s == nil {
    fmt.Println("nil slice")
}
```

---

map（映射）

初始化 map

```
m := map[string]int{
    "Alice": 25,
    "Bob":   30,
}
```

添加/修改键值对

```
m["Charlie"] = 35
```

删除键值对

```
delete(m, "Bob")
```

查询键是否存在

```
value, exists := m["Alice"]
if exists {
    fmt.Println(value)
}

if value, exists := m["Alice"];exists {//注意分号
    fmt.Println(value)
}
```

题目案例

题目 2：字典频率统计

描述：
给定一个字符串，统计每个字符出现的次数，并按频率降序输出前 k 个字符及其次数。若频率相同，则按字典序排列。
输入：`s = "abracadabra", k = 3`
输出：`[('a', 5), ('b', 2), ('r', 2)]`

难度：★★★☆☆
参考答案：

```
import (
    "container/heap"
    "sort"
)

func topKFrequent(s string, k int) []pair {
    freq := make(map[rune]int)
    for _, c := range s {
        freq[c]++
    }

    pairs := make([]pair, 0, len(freq))
    for c, cnt := range freq {
        pairs = append(pairs, pair{c, cnt})
    }

    sort.Slice(pairs, func(i, j int) bool {
        if pairs[i].count == pairs[j].count {
            return pairs[i].char < pairs[j].char
        }
        return pairs[i].count > pairs[j].count
    })

    return pairs[:k]
}

type pair struct {
    char  rune
    count int
}
```

7-8 字符串去重排序

分数 10

全屏浏览切换布局

作者 刘琦

单位 天津城建大学

输入一个非空字符串，去除重复的字符后，从小到大排序，输出为一个新字符串。

```
package main

import (
    "bufio"  // 用于高效读取输入
    "fmt"    // 格式化输入输出
    "os"     // 操作系统交互，如读取标准输入
    "sort"   // 提供排序功能
    "strings" // 字符串处理函数
```

```go
)

func main() {
    // 读取输入字符串并去除首尾空白
    reader := bufio.NewReader(os.Stdin)
    input, _ := reader.ReadString('\n')
    input = strings.TrimSpace(input)

    // 使用map去重：map的键具有唯一性，值设为true即可
    charMap := make(map[rune]bool)
    for _, char := range input {
        charMap[char] = true // 重复的字符会自动覆盖
    }

    // 将map的键（不重复的字符）转换为rune切片
    chars := make([]rune, 0, len(charMap))
    for char := range charMap {
        chars = append(chars, char)
    }

    // 对字符切片进行升序排序（按Unicode码点）
    sort.Slice(chars, func(i, j int) bool {
        return chars[i] < chars[j]
    })

    // 将排序后的rune切片转换为字符串并输出
    result := string(chars)
    fmt.Println(result)
}
```

智能合约

简单合约

```yaml
pragma solidity ^0.8.0;

contract Storage {
    uint256 number;

    function store(uint256 num) public {
        number = num;
    }

    function retrieve() public view returns (uint256) {
        return number;
    }
}
```

Solidity 提供了四种不同的函数可见性修饰符：

- **public**：可以从外部和合约内部访问。
- **private**：只能从合约内部访问。
- **internal**：只能从当前合约及其派生合约访问。
- **external**：只能从外部访问，不能从合约内部调用。

```
contract Example {
    function publicFunc() public view returns (uint) { return 1; }
    function privateFunc() private pure returns (uint) { return 2; }
    function internalFunc() internal pure returns (uint) { return 3; }
    function externalFunc() external pure returns (uint) { return 4; }
}
```

olidity 还提供了三种函数状态可变性修饰符：

- **view**：声明该函数不会修改状态。
- **pure**：声明该函数既不读取也不修改状态。
- **payable**：允许函数接收以太币。

```
contract Example {
    function viewFunc() public view returns (uint) { return 1; }
    function pureFunc() public pure returns (uint) { return 2; }
    function payableFunc() public payable returns (uint) { return msg.value; }
}
```

简易程序案例

```
// SPDX-License-Identifier: MIT  // 指定合约的开源许可证为 MIT
pragma solidity ^0.8.0;  // 指定编译此合约所需的 Solidity 版本

contract SimpleToken {
    // 代币的基本信息
    string public name = "SimpleToken";  // 代币名称
    string public symbol = "ST";  // 代币符号
    uint8 public decimals = 18;  // 代币精度（18 位小数，与 ETH 相同）
    uint256 public totalSupply;  // 代币总供应量
    mapping(address => uint256) public balanceOf;  // 地址到余额的映射

    // 合约构造函数，部署时执行
    constructor(uint256 _initialSupply) {
        // 计算初始总供应量（考虑小数位）
        totalSupply = _initialSupply * (10 ** uint256(decimals));
        // 将初始代币全部分配给合约部署者
        balanceOf[msg.sender] = totalSupply;
    }

    // 代币转账函数
    function transfer(address _to, uint256 _value) public returns (bool success)
{
        // 检查发送者余额是否足够
```

```
        require(balanceOf[msg.sender] >= _value, "余额不足");
        // 从发送者账户扣除代币
        balanceOf[msg.sender] -= _value;
        // 向接收者账户添加代币
        balanceOf[_to] += _value;
        return true;
    }

    // 铸造新代币的函数（增发）
    function mint(address _to, uint256 _value) public {
        // 权限检查：仅允许管理员地址（0x1）铸造代币
        require(msg.sender == address(0x1), "只有管理员可以铸造代币");
        // 增加总供应量
        totalSupply += _value;
        // 向指定地址添加代币
        balanceOf[_to] += _value;
    }
}
```

4.1 基本类型

整数类型：

- `uint` 和 `int`：分别表示无符号整数和有符号整数。
  - `uint8`, `uint16`, ..., `uint256`：从 8 到 256 位的无符号整数。
  - `int8`, `int16`, ..., `int256`：从 8 到 256 位的有符号整数。

```
uint public myUint = 10;
int public myInt = -10;
```

地址类型：

- `address`：用于存储以太坊账户地址。
  - `address payable`：可接收以太币的地址。

```
address public owner;
address payable public receiver;
```

布尔类型：

- `bool`：用于存储布尔值（true 或 false）。

```
bool public isActive = true;
```

字符串类型：

- `string`：用于存储任意长度的字符串。

```
string public name = "Alice";
```

4.2 数组与映射

数组：

- 固定大小数组：`type[size]`
- 动态大小数组：`type[]`

```
uint[3] public fixedArray = [1, 2, 3];
uint[] public dynamicArray = [1, 2, 3, 4];
```

映射：

- `mapping(keyType => valueType)`：类似于哈希表或字典。

```
mapping(address => uint) public balances;
```